UNIVERSIDAD NACIONAL DE TUCUMÁN

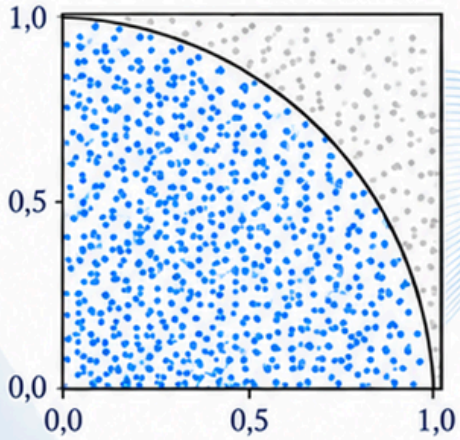
UNIVERSIDAD NACIONAL DE TUCUMÁN
facet
FACULTAD DE CIENCIAS EXACTAS Y TECNOLOGÍA

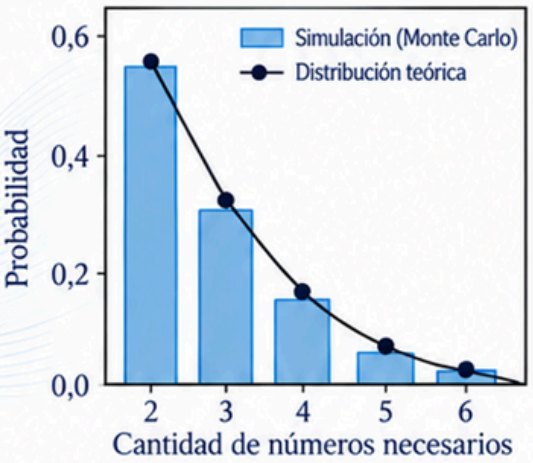
COMPUTACIÓN CIENTÍFICA

EVALUACIÓN PARCIAL 1

Estimación de Constantes Matemáticas (π y e)

IMPLEMENTACIÓN MEDIANTE EL MÉTODO DE MONTE CARLO Y CÓMPUTO DE ALTO RENDIMIENTO (MPI)





Grupo:

Campos, Nazaret

Cano, Juan

Diaz, Daniela

2026

1. Introducción:

Los métodos de Monte Carlo constituyen un conjunto de técnicas numéricas basadas en la generación de muestras aleatorias para obtener soluciones aproximadas a diferentes problemas matemáticos y computacionales. Su aplicación permite estimar cantidades de interés mediante la repetición de experimentos aleatorios y el análisis estadístico de los resultados obtenidos.

En este proyecto se estudia la estimación de dos constantes matemáticas fundamentales: π y e. Para aproximar π, se utiliza un procedimiento geométrico basado en la generación de puntos aleatorios dentro de un cuadrado y el cálculo de la proporción de puntos que pertenecen a un círculo inscrito. Por otro lado, para aproximar e, se emplea un procedimiento probabilístico que consiste en generar y sumar números aleatorios uniformemente distribuidos entre 0 y 1 hasta superar la unidad, registrando la cantidad de valores necesarios en cada experimento.

Debido a que ambos procedimientos requieren realizar numerosas simulaciones independientes, resultan adecuados para aplicar técnicas de computación paralela. Por este motivo, se propone desarrollar una implementación en lenguaje C utilizando MPI (*Message Passing Interface*), distribuyendo las simulaciones entre diferentes procesos y combinando posteriormente los resultados parciales para obtener las estimaciones.

Finalmente, se analizará el comportamiento de las aproximaciones al aumentar la cantidad de simulaciones, evaluando su convergencia y los errores obtenidos respecto de los valores teóricos. También se estudiará el rendimiento de la implementación paralela mediante la medición de tiempos de ejecución y el cálculo de métricas como el *speedup* y la eficiencia, con el propósito de identificar las ventajas y limitaciones de la paralelización.

2. Objetivos

El objetivo de este proyecto es estudiar y aplicar los métodos de Monte Carlo para obtener aproximaciones de las constantes matemáticas π y e, analizando los fundamentos probabilísticos que permiten realizar dichas estimaciones. A partir de diferentes cantidades de simulaciones, se buscará evaluar el comportamiento de las aproximaciones obtenidas, estudiar su convergencia hacia los valores de referencia y analizar los errores asociados a cada estimación.

Asimismo, se propone desarrollar y evaluar una implementación paralela de los algoritmos utilizando MPI (*Message Passing Interface*), distribuyendo las simulaciones entre diferentes procesos para analizar su rendimiento computacional. Para ello, se realizarán múltiples ejecuciones variando la cantidad de procesos y, cuando sea posible, utilizando diferentes

sistemas de cómputo. Se registrarán los tiempos de ejecución y se calcularán métricas como el *speedup* y la eficiencia. Finalmente, se analizarán los resultados obtenidos para identificar las ventajas y limitaciones de la solución implementada y proponer posibles mejoras.

3. Fundamentación teórica y modelado matemático

3.1. Métodos de Monte Carlo

Los métodos de Monte Carlo constituyen un conjunto de técnicas numéricas que utilizan el muestreo aleatorio repetido para obtener aproximaciones a problemas matemáticos y computacionales. Estos métodos comenzaron a desarrollarse durante la década de 1940, principalmente a partir de los trabajos de Stanislaw Ulam, John von Neumann y Nicholas Metropolis, en el contexto de investigaciones realizadas en el Laboratorio Nacional de Los Álamos. Desde entonces, su utilización se ha extendido a diferentes áreas de la ciencia, la ingeniería y la computación.

El fundamento probabilístico de los métodos de Monte Carlo basados en muestras independientes se encuentra en la ley de los grandes números, la cual establece que, bajo determinadas condiciones, el promedio de una cantidad creciente de observaciones aleatorias independientes tiende a aproximarse al valor esperado de la variable estudiada. Por este motivo, al aumentar el número de simulaciones, las estimaciones tienden a estabilizarse alrededor del valor que se desea calcular, aunque pueden presentar fluctuaciones entre diferentes ejecuciones.

Una de las principales características de estos métodos es que permiten abordar problemas mediante experimentos aleatorios que pueden repetirse de forma independiente. Esta propiedad resulta especialmente útil en computación paralela, ya que las simulaciones pueden distribuirse entre diferentes procesos para reducir el tiempo de ejecución.

3.2. Estimación de π mediante Monte Carlo

El número Pi (π) es una constante matemática que representa la relación entre la longitud de una circunferencia y su diámetro. Es un número irracional y su valor comienza aproximadamente como:

$$\pi = 3.141592653589793\ldots$$

Una de las formas de estimar π mediante Montecarlo consiste en aproximar el valor del área de una figura geométrica conocida (el círculo) mediante el muestreo aleatorio de puntos.
Imaginemos un plano cartesiano donde dibujamos un **círculo unitario** (de radio = 1) centrado en el origen (0,0). Este círculo está inscrito perfectamente dentro de un cuadrado cuyo lado mide 2 (va desde -1 a 1 en ambos ejes)

Región de muestreo
Cuadrado: $[-1, 1] \times [-1, 1]$
Área: $A_{\text{cuadrado}} = 2^2 = 4$

Región de interés
Círculo unitario: $x^2 + y^2 \leq 1$
Área: $A_{\text{círculo}} = \pi$

Se generan N puntos aleatorias (x,y) uniformemente distribuidos en el cuadrado y se cuentan cuantos caen dentro del círculo:

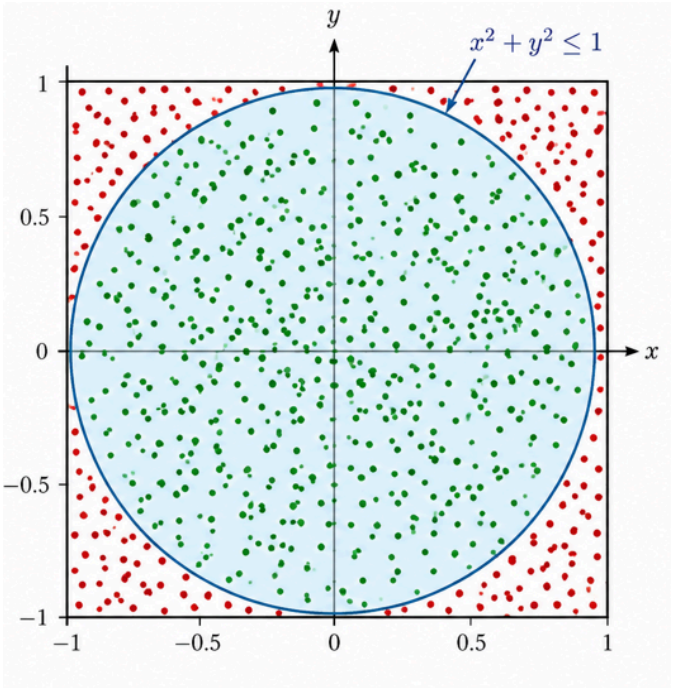
- Punto dentro del círculo $(x^2 + y^2 \leq 1)$
- Punto fuera del círculo $(x^2 + y^2 > 1)$

La probabilidad (P) de que un punto caiga dentro del círculo es equivalente a la proporción que ocupa el área del círculo respecto al área total del cuadrado:

$$P(dentro) = \frac{Acirculo}{Acuadrado} = \frac{\pi}{4}$$

Si Ndentro es la cantidad de puntos que caen dentro del círculo, entonces:

$$\frac{N_{\text{dentro}}}{N} \approx \frac{\pi}{4} \Rightarrow \hat{\pi} = 4\frac{N_{\text{dentro}}}{N}$$



A medida que aumenta el número de puntos generados, la proporción observada tiende a estabilizarse alrededor de la probabilidad teórica y, en consecuencia, la estimación de π tiende a aproximarse a su valor real. Sin embargo, debido al carácter aleatorio del método, pueden presentarse fluctuaciones entre diferentes ejecuciones.

3.3. Estimación de e mediante Monte Carlo

El número e, también llamado **número de Euler,** es una constante matemática:

$$e = 2.718281828459045\ldots$$

Es un número irracional, igual que π, por lo que tiene infinitos decimales no periódicos.
Aparece principalmente en fenómenos relacionados con **crecimiento y decrecimiento exponencial**, logaritmos naturales, ecuaciones diferenciales, probabilidad, estadística, interés compuesto, etc.
Una de sus representaciones matemáticas es mediante la serie:

$$e = \sum_{n=0}^{\infty} \frac{1}{n!} \qquad e = 1 + 1 + \frac{1}{2!} + \frac{1}{3!} + \frac{1}{4!} + \cdots$$

Para estimar e mediante Monte Carlo se utiliza un procedimiento probabilístico que consiste en generar números aleatorios independientes, uniformemente distribuidos entre 0 y 1, y sumarlos sucesivamente hasta que la suma acumulada supere 1. En cada experimento se registra la cantidad de números necesarios para alcanzar esa condición.

El procedimiento se repite una gran cantidad de veces, obteniendo diferentes cantidades de números generados. Si M representa el total de experimentos y N_i la cantidad de números necesarios en el experimento i, la estimación se calcula mediante:

$$\hat{e} = \frac{1}{M} \sum_{i=1}^{M} N_i$$

El fundamento matemático de este procedimiento se encuentra en que la probabilidad de que la suma de n variables aleatorias independientes, uniformemente distribuidas entre 0 y 1, sea menor o igual que 1, es:

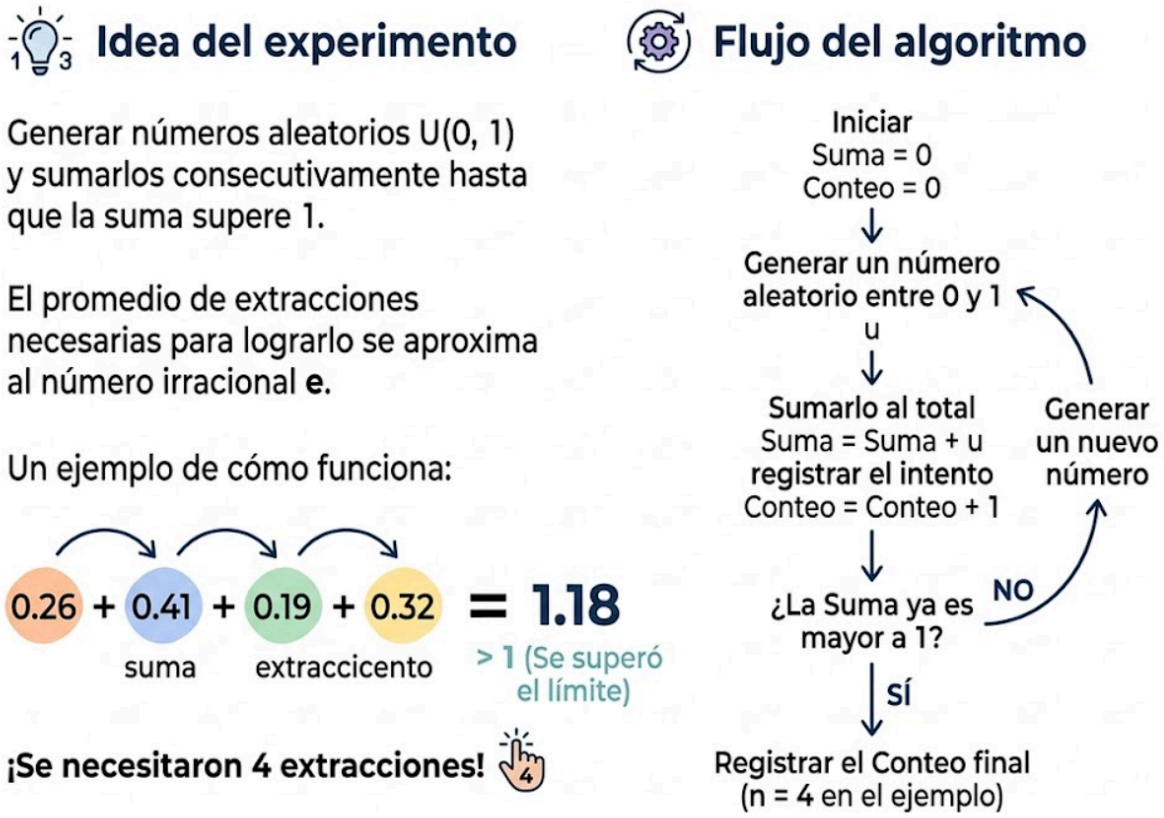
$$P(U_1 + U_2 + \cdots + U_n \leq 1) = \frac{1}{n!}$$

A partir de esta propiedad puede demostrarse que el valor esperado de la cantidad de números necesarios para superar 1 coincide con la serie que define al número *e*:

$$E[N] = \sum_{n=0}^{\infty} \frac{1}{n!}$$

Por lo tanto: $\boxed{E[N] = e}$

En consecuencia, al aumentar la cantidad de experimentos, el promedio de las cantidades registradas tiende a aproximarse al valor de *e*, aunque pueden presentarse fluctuaciones aleatorias entre diferentes ejecuciones.



3.4. Convergencia y análisis del error

Los métodos de Monte Carlo producen estimaciones aproximadas que dependen de los valores aleatorios generados durante cada ejecución. Por este motivo, es necesario analizar cómo se comportan las estimaciones a medida que aumenta la cantidad de simulaciones y evaluar su diferencia respecto de los valores teóricos.

El comportamiento matemático de ambos estimadores de Monte Carlo está gobernado por:

1. **La ley de los grandes números**
2. **Teorema del límite central (TLC)**

La **ley de los grandes números**, establece que, bajo determinadas condiciones, el promedio de una cantidad creciente de observaciones aleatorias independientes converge hacia su valor esperado. En este proyecto, esto permite justificar que las estimaciones de π y e tienden a aproximarse a sus respectivos valores teóricos al aumentar el número de simulaciones.

El **teorema del límite central (TLC)** permite describir las fluctuaciones estadísticas de estos estimadores. Para los procedimientos utilizados, la desviación estándar de la estimación disminuye proporcionalmente a: $\frac{1}{\sqrt{N}}$

donde N representa la cantidad de simulaciones. Esto implica que, para reducir aproximadamente a la mitad la desviación estándar, es necesario cuadruplicar la cantidad de muestras. Sin embargo, debido al carácter aleatorio del método, el error observado no necesariamente disminuye en cada ejecución individual.

Para evaluar la precisión de las aproximaciones obtenidas se utilizarán dos medidas:

1. **Error absoluto**
2. **Error relativo porcentual.**

El **error absoluto** representa la diferencia en valor absoluto entre la estimación y el valor teórico:

$$E_{\text{abs}} = |V_{\text{estimado}} - V_{\text{teórico}}|$$

El **error relativo porcentual** expresa esa diferencia en relación con el valor teórico, permitiendo comparar los errores de distintas estimaciones:

$$E_{\text{rel}}(\%) = \frac{|V_{\text{estimado}} - V_{\text{teórico}}|}{|V_{\text{teórico}}|} \times 100$$

Estas medidas se aplicarán a las aproximaciones de π y e obtenidas mediante diferentes cantidades de simulaciones, con el objetivo de estudiar su convergencia y analizar la evolución del error.

Costo computacional y precisión de las estimaciones

Una de las principales limitaciones de los métodos de Monte Carlo es que la reducción del error estadístico requiere aumentar considerablemente la cantidad de simulaciones. Para los estimadores utilizados en este proyecto, la desviación estándar disminuye proporcionalmente a $1/\sqrt{N}$, donde (N) representa el número de simulaciones realizadas.

La consecuencia directa del factor $1/\sqrt{N}$ es una ley de rendimientos decrecientes severa:

- Para reducir el margen de error a la mitad (1/2), es estrictamente necesario multiplicar la muestra por $2^2 = 4$ veces.
- Para ganar un dígito decimal exacto de precisión (reducir el error en un factor de 10), se requiere generar: $10^2 = 100$ veces más muestras.
- Para ganar dos dígitos decimales: $10^4 = 10{,}000$ veces más muestras.
- Para ganar tres dígitos decimales: $10^6 = 1{,}000{,}000$ de veces más muestras.

Esta lentitud intrínseca es la justificación matemática absoluta de la Computación de Alto Rendimiento (HPC): dado que una computadora mononúcleo tardaría días o semanas en procesar las decenas de miles de millones de muestras necesarias para ganar estabilidad

3.5. Generación de números pseudoaleatorios (PRNG) y Ergodicidad

Los métodos de Monte Carlo requieren generar valores aleatorios para realizar las simulaciones. En computación, estos valores se obtienen habitualmente mediante generadores de números pseudoaleatorios, que utilizan algoritmos deterministas para producir secuencias con propiedades estadísticas similares a las de los números aleatorios.

Una característica importante de estos generadores es que utilizan un valor inicial denominado **semilla** (*seed*). Cuando se emplea el mismo generador con la misma semilla, es

posible reproducir la secuencia obtenida, lo cual resulta útil para verificar resultados y repetir experimentos bajo condiciones controladas.

Existen diferentes algoritmos para generar secuencias de números pseudoaleatorios, entre los cuales se destacan los generadores congruenciales lineales (LCG) y Mersenne Twister (MT19937).

1. **Generadores Congruenciales Lineales (LCG):** Implementados en rand() de C, producen una secuencia de números enteros mediante una relación de recurrencia: Xn+1 = (a Xn + c) mod m, donde (X_n) representa el valor actual de la secuencia, (a) es un multiplicador, (c) es un incremento y (m) es el módulo.
   Tienen ciclo finito (2^{31}-1) y sufren del teorema de Marsaglia. Requieren gestión estricta de semillas.

2. **Mersenne Twister (MT19937):** Utilizado en Python (random y NumPy), con ciclo de 2^{19937}-1 y buenas propiedades de equidistribución en hasta 623 dimensiones.

Ergodicidad: La secuencia de valores generados debe cubrir de manera uniforme todo el dominio continuo sin agruparse en bandas ni formar vacíos artificiales.

4. Arquitectura y Diseño de la Solución Paralela en C + MPI

MPI (Message Passing Interface) es un estándar para desarrollar programas paralelos en los que distintos procesos pueden comunicarse e intercambiar datos. Cada proceso tiene su propio espacio de memoria y se identifica mediante un número llamado rango (rank).

4.1. Paradigma de Paralelismo y Descomposición del Problema

Los métodos de Monte Carlo utilizados en este proyecto constituyen un ejemplo de problemas *vergonzosamente paralelos* (*embarrassingly parallel*), debido a que las simulaciones pueden ejecutarse de manera independiente y requieren una comunicación mínima entre los procesos.

- **Cero Dependencia de Datos:** cada par de coordenadas (x, y) utilizado para estimar π y cada experimento de suma acumulada para estimar e pueden procesarse de forma aislada, sin depender de los resultados calculados por otros procesos.
- **Modelo SPMD (*Single Program, Multiple Data*):** todos los procesos ejecutan el mismo programa, pero cada uno trabaja con su propio conjunto de simulaciones.

Particionamiento estático y balanceo de carga

Dado un total de N simulaciones y P procesos concurrentes, el programa utiliza un particionamiento estático que asigna a cada proceso la misma cantidad de experimentos: $N_{local} = \lfloor N / P \rfloor$

Al demandar cada iteración una cantidad casi idéntica de ciclos de instrucción, el tiempo de cálculo es homogéneo en todos los núcleos ($t_{comp}^{(0)} \approx t_{comp}^{(1)} \approx ... \approx t_{comp}^{(P-1)}$), alcanzando un balanceo de carga óptimo sin necesidad de esquemas dinámicos maestro-trabajador.

4.2. Implementación en C con MPI

La solución paralela de alto rendimiento fue implementada en C utilizando el estándar MPI (Message Passing Interface) para memoria distribuida. El código fuente se encuentra en el archivo independiente **MonteCarlo_Aproximacion_PI_y_E.c**

Flujo operativo del algoritmo paralelo

1. **MPI_Init(&argc, &argv)** inicializa el comunicador MPI_COMM_WORLD, es decir se inicializa el entorno de ejecución paralelo.
2. **MPI_Comm_rank** obtiene el identificador de cada proceso my_rank (0 ≤ my_rank < P) dentro del comunicador.
3. **MPI_Comm_size** determina el número total de procesos participantes, representado por P.
4. **N_local = N / P**, partición local, calculamos la cantidad de experimentos asignados a cada proceso.
5. **srand(time(NULL) + my_rank * 1013)** Inicialización del generador pseudoaleatorio, cada proceso utiliza una semilla calculada a partir del tiempo actual y su rango
6. **MPI_Wtime()** inicio de la medición temporal, el proceso de rango 0 registra el tiempo.
7. **Ejecución de las simulaciones locales:** cada proceso realiza los experimentos correspondientes a ambas constantes:
   - Para π, **calcular_puntos_circulo(iteraciones_por_proceso)** genera puntos aleatorios y contabiliza aquellos que pertenecen al círculo unitario.
   - Para e, **calcular_extracciones_euler(iteraciones_por_proceso)** realiza experimentos de sumas acumuladas y contabiliza la cantidad de extracciones necesarias para superar la unidad.

8. Reducción de resultados: mediante **MPI_Reduce**, los procesos combinan sus resultados parciales utilizando la operación MPI_SUM. Se realizan dos reducciones: una para obtener la cantidad total de puntos dentro del círculo y otra para acumular las extracciones utilizadas en los experimentos de Euler.

   - MPI_Reduce(&puntos_pi_local, &puntos_pi_total, 1, MPI_LONG_LONG, MPI_SUM, 0, MPI_COMM_WORLD)

   - MPI_Reduce(&extracciones_e_local, &extracciones_e_total, 1, MPI_LONG_LONG, MPI_SUM, 0, MPI_COMM_WORLD)

9. El proceso coordinador (rango 0) calcula las **aproximaciones finales** a partir de los resultados acumulados:

   $\hat{\pi} = 4 \times$ *(puntos dentro del círculo / simulaciones ejecutadas)*

   $\hat{e}$ *= extracciones totales / experimentos ejecutados*

   Posteriormente, registra el tiempo de ejecución y calcula los errores de las aproximaciones respecto de los valores teóricos.

10. **MPI_Finalize()** finaliza el entorno MPI, libera todos los recursos del comunicador.

4.3. Gestión Crítica de Semillas Pseudoaleatorias Concurrentes

La Trampa del Determinismo Clonado

Si todos los procesos MPI inicializan su generador mediante srand(time(NULL)), se comete un error metodológico grave:
Cuando mpirun o mpiexec lanza P procesos en un clúster, todos inician en la misma fracción de segundo del reloj del sistema. La llamada time(NULL) devolverá exactamente el mismo número entero de timestamp a los P procesos.
Como el generador LCG es completamente determinista, los P procesos generarán exactamente la misma secuencia de números pseudoaleatorios.
En consecuencia los P procesadores evaluarán los mismos puntos y las mismas sumas. No se estarán generando N datos independientes, sino repitiendo los mismos N/P datos P veces. La varianza no se reduce y se desperdicia por completo la inversión en procesadores paralelos.

Solución Implementada: Perturbación Modular por Rango

Para reducir el riesgo de generar secuencias idénticas, el programa utiliza una semilla que combina el tiempo actual con el rango de cada proceso MPI:

```
C/C++
srand((unsigned int)(time(NULL) + my_rank * 1013));
```

Al incorporar el multiplicador primo 1013 sobre el rango del proceso, las semillas de cada núcleo se distancian ampliamente en el espacio de estados del generador, explorando trayectorias ortogonales e independientes.

4.4. Prototipo y Comparativa: C + MPI vs. Python Multiproceso

Para el análisis interactivo y la generación de gráficos se desarrolló el script complementario simulaciones_y_graficos.py en Python.

- **El Cuello de Botella del GIL en Python:** El cerrojo global del intérprete (Global Interpreter Lock - GIL) impide que los hilos nativos de Python (threading) utilicen múltiples núcleos de CPU en tareas de cálculo numérico intensivo.
- **Solución Multiproceso:** Para paralelizar en Python se emplea concurrent.futures.ProcessPoolExecutor(), creando procesos independientes del sistema operativo que eluden el GIL.

En este proyecto se optó por desarrollar la implementación paralela en C utilizando MPI, debido al control que proporciona sobre la ejecución de los procesos y a su utilización en entornos de computación de alto rendimiento. Por otra parte, Python se utilizó para realizar simulaciones de referencia, analizar resultados y generar representaciones gráficas.

Dimensión de Análisis	Implementación C + MPI (MonteCarlo_Aproximacion_PI_y_E.c)	Implementación Python (simulaciones_y_graficos.py)
Nivel de Ejecución	Binario nativo compilado a código máquina con optimizaciones -O3.	Código interpretado sobre CPython con aceleración en C (NumPy/math).
Escalabilidad de Red	Memoria compartida y memoria distribuida en clústeres multi-nodo.	Exclusivo para memoria compartida en una sola máquina local.
Consumo de Memoria RAM	Escalar O(1) por proceso (evaluación al vuelo en bucles).	Vectorial O(N) si se usan arreglos masivos en memoria.
Sobrecarga de Comunicación	Mínima (llamadas nativas de socket/red con MPI_Reduce).	Elevada (serialización de objetos con pickle e inter-process pipes).
Propósito en el Proyecto	Cómputo de producción masiva para N ≥ 10^7 hasta 10^11 en clúster.	Prototipado, validación rápida de algoritmos y renderizado de gráficos.

5. Metodología Experimental y Entornos de Cómputo

5.1. Infraestructura 1: Estación de Trabajo Personal (Notebook)

- **Arquitectura:** Procesador multinúcleo x86_64 con extensiones AVX2.
- **Configuración de Núcleos:** 4 núcleos físicos y 8 hilos lógicos (SMT / Hyper-Threading).
- **Jerarquía de Memoria:** Memoria RAM compartida DDR4 y caché L3 unificada de 8 MB.
- **Rango de Concurrencia:** $P \in \{1, 2, 3, 4, 6\}$ procesos.
- **Rango Muestral:** Cargas progresivas desde N = 10^1 hasta N = 8 × 10^7 puntos (80 millones).

5.2. Infraestructura 2: Clúster de Supercómputo FaMAF (UNC)

- **Capacidad de Cómputo:** Nodo de alto rendimiento equipado con 32 procesadores dedicados (CPUs) en la Facultad de Matemática, Astronomía y Física de la Universidad Nacional de Córdoba.
- **Entorno de Red:** Sistema operativo Linux para servidores sin interfaz gráfica (headless), con soporte nativo de baja latencia para MPI.
- **Carga de Estrés Extremo:** Carga masiva constante fijada en cien mil millones de puntos:

  $N = 10 \times 10^{10} = 10^{11}$ *puntos*
- **Rango de Procesos Evaluados:**

  $P \in \{1, 2, 3, 4, 5, 6, 7, 8, 12, 14, 16, 18, 24, 26, 28\}$

5.3. Protocolo de Medición y Eliminación de Ruido Experimental

- **1. La Falsedad de la Media Simple:** En sistemas operativos multitarea, las interrupciones del kernel, la recolección de memoria y los demonios de red introducen retrasos aleatorios. La media aritmética se ve severamente afectada por estos valores atípicos (outliers), distorsionando el cálculo del SpeedUp.
- **2. Uso Obligatorio de la Mediana Muestral:** Siguiendo la directiva formal de la cátedra de Métodos Numéricos II de FACET, para cada configuración de procesos se realizaron múltiples ejecuciones independientes y se computó la mediana estadística de los tiempos.
- **3. Calentamiento de Caché (Warm-up):** Se ejecutó una corrida preliminar no cronometrada para asegurar la precarga de bibliotecas dinámicas en la memoria RAM y evitar penalizaciones de fallos de página (page faults).
- **4. Aislamiento del Reloj de Pared:** Las marcas de tiempo MPI_Wtime() envolvieron strictly la fase de cálculo local y la llamada colectiva MPI_Reduce, descartando intencionalmente los tiempos de inicialización de MPI y de salida por consola (printf).

6. Análisis de Resultados de Aproximación, Error y Convergencia

6.1. Resultados Empíricos de Simulación y Convergencia Numérica

A continuación se presentan los resultados numéricos obtenidos al evaluar ambas constantes sobre ocho órdenes de magnitud (N = 10^1 hasta N = 8 × 10^7 muestras):

Tabla 1: Resultados Empíricos de la Estimación de π (N = 10^1 a 8 × 10^7)

Valor teórico analítico: π ≈ 3.141592653589793…

Muestras (N)	Aproximación Empírica π̂	Error Absoluto ε_abs	Error Relativo (%)	Tiempo de Cómputo (s)
10^1	3.600000	0.458407	14.5916%	0.0000
10^2	3.360000	0.218407	6.9521%	0.0001
10^3	3.148000	0.006407	0.2040%	0.0001
10^4	3.122800	0.018793	0.5982%	0.0004
10^5	3.149080	0.007487	0.2383%	0.0033
10^6	3.141640	0.000047	0.0015%	0.0329
10^7	3.141820	0.000227	0.0072%	0.3259
8 × 10^7	3.141656	0.000063	0.0020%	2.6100

Tabla 2: Resultados Empíricos de la Estimación de Euler e (N = 10^1 a 8 × 10^7)

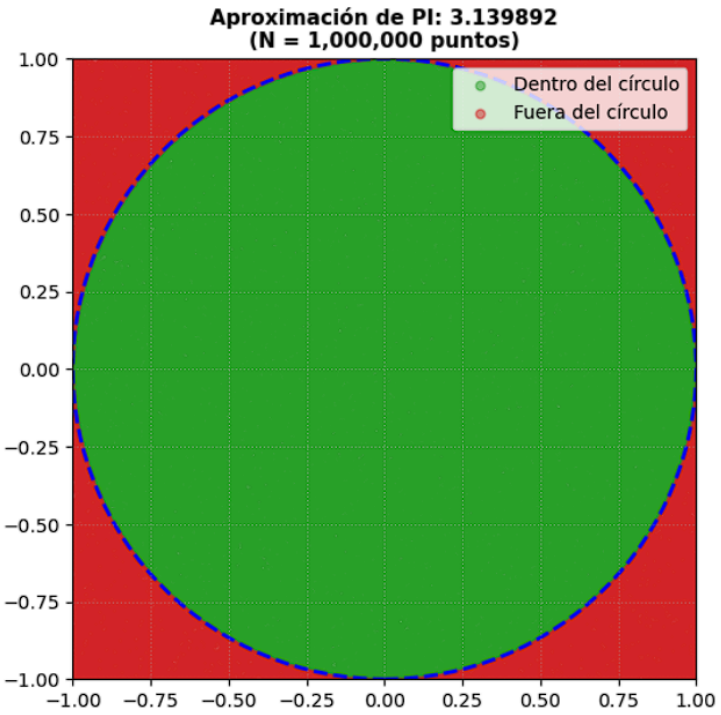
Valor teórico analítico: e ≈ 2.718281828459045…

Muestras (N)	Aproximación Empírica ê	Error Absoluto ε_abs	Error Relativo (%)	Tiempo de Cómputo (s)
10^1	2.600000	0.118282	4.3513%	0.0000
10^2	2.770000	0.051718	1.9026%	0.0001
10^3	2.750000	0.031718	1.1668%	0.0001
10^4	2.709500	0.008782	0.3231%	0.0004
10^5	2.713650	0.004632	0.1704%	0.0033
10^6	2.719123	0.000841	0.0309%	0.0329
10^7	2.718568	0.000286	0.0105%	0.3259
8×10^7	2.718314	0.000032	0.0012%	2.6100

6.2. Catálogo de Gráficos e Indicaciones de Inserción

A continuación se detallan las especificaciones técnicas de los gráficos requeridos para la memoria y la defensa oral del parcial:

Gráfico 1:

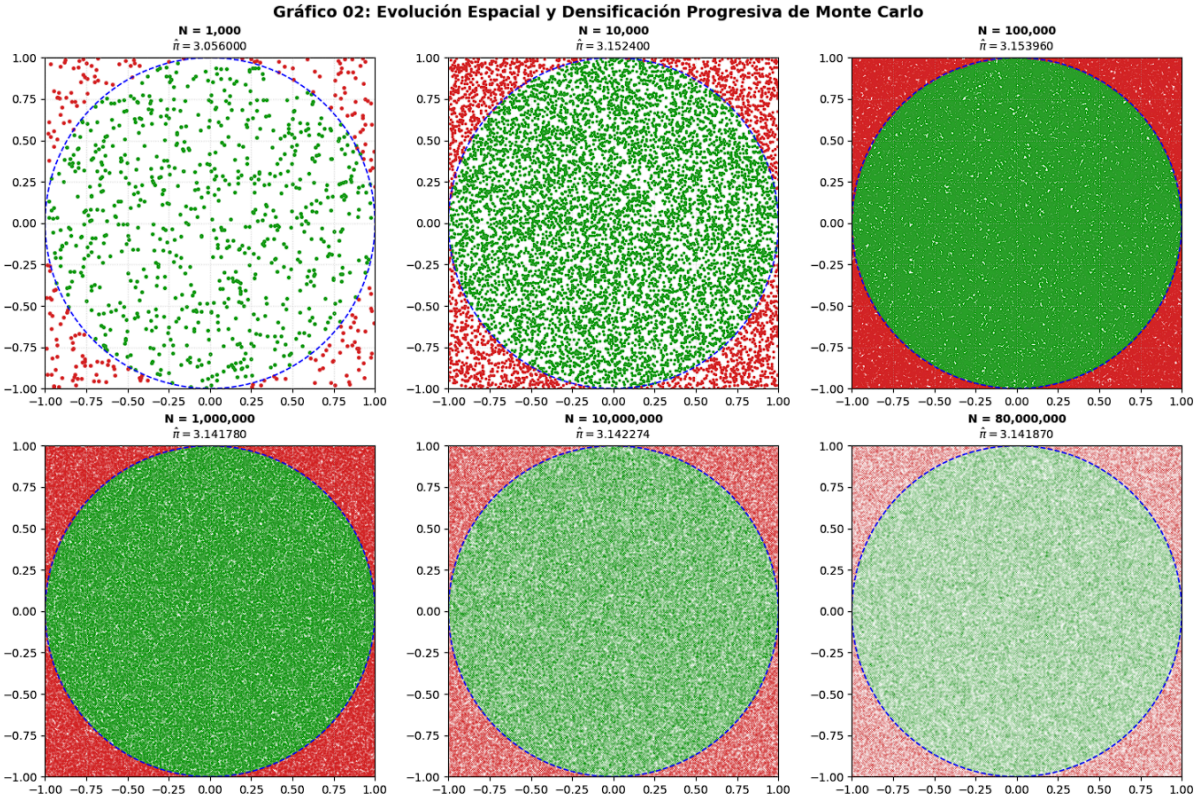


Tipo de Gráfico: Diagrama de dispersión cartesiana bidimensional con relación de aspecto cuadrada 1:1.

Contenido Visual: Marco cuadrado acotado en [-1.0, 1.0] × [-1.0, 1.0]. En el interior, los puntos que cumplen $x^2 + y^2 \leq 1.0$ forman un disco sólido continuo de color verde esmeralda delimitado por una circunferencia azul discontinua de radio unitario R = 1. En las cuatro esquinas exteriores ($x^2 + y^2 > 1.0$), los puntos se visualizan en color rojo. Muestra 10^6 puntos con el título indicando la aproximación de π.

Interpretación Física/Numérica: Evidencia la ergodicidad y homogeneidad del generador pseudoaleatorio en el plano bidimensional, mostrando visualmente que la relación entre la masa verde y la superficie total converge de forma exacta a $\pi/4 \approx 0.785398$.

Gráfico 2:

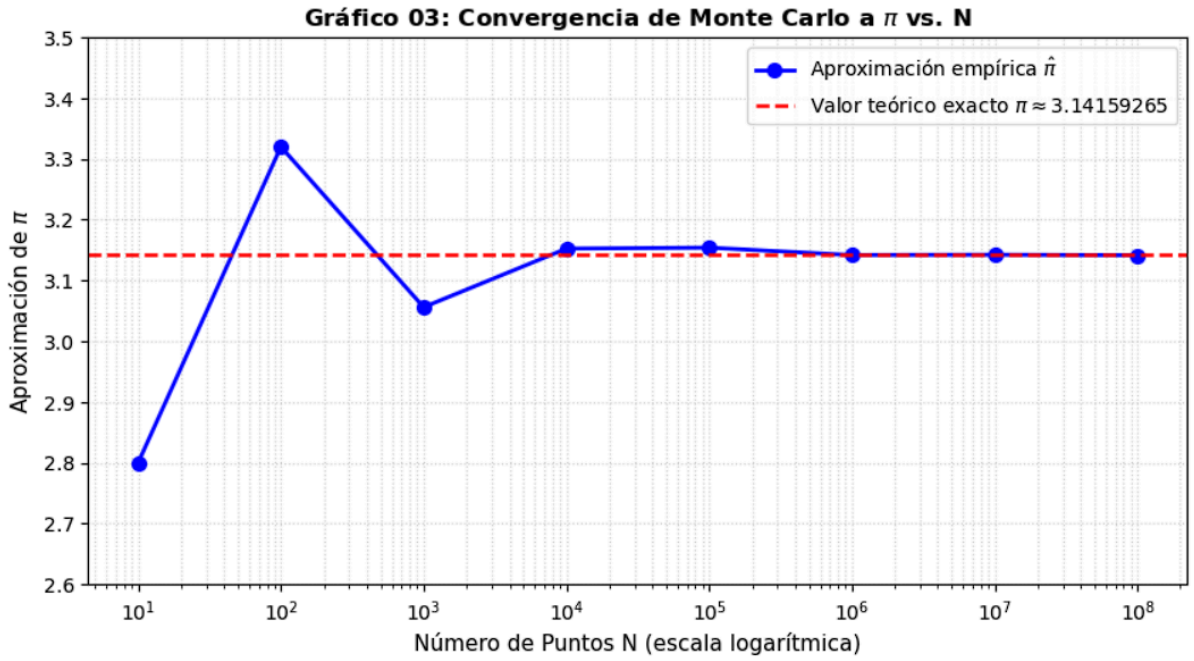


Tipo de Gráfico: Panel matricial de 6 capturas espaciales progresivas (2 × 3).

Contenido Visual: Muestra la densificación gradual del plano cartesiano para N ∈ {10^3, 10^4, 10^5, 10^6, 10^7, 8×10^7}. Se observa la transición desde puntos aislados con predominio de fondo blanco (N=10^3), pasando por la definición del contorno circular (N=10^4 y 10^5), hasta la saturación cromática sólida y perfecta (N=8×10^7).

Interpretación Física/Numérica: Representa la evolución visual del principio de los grandes números: a medida que el espacio de estados se cubre exhaustivamente, la varianza estocástica de la frontera desaparece.

Gráfico 3:

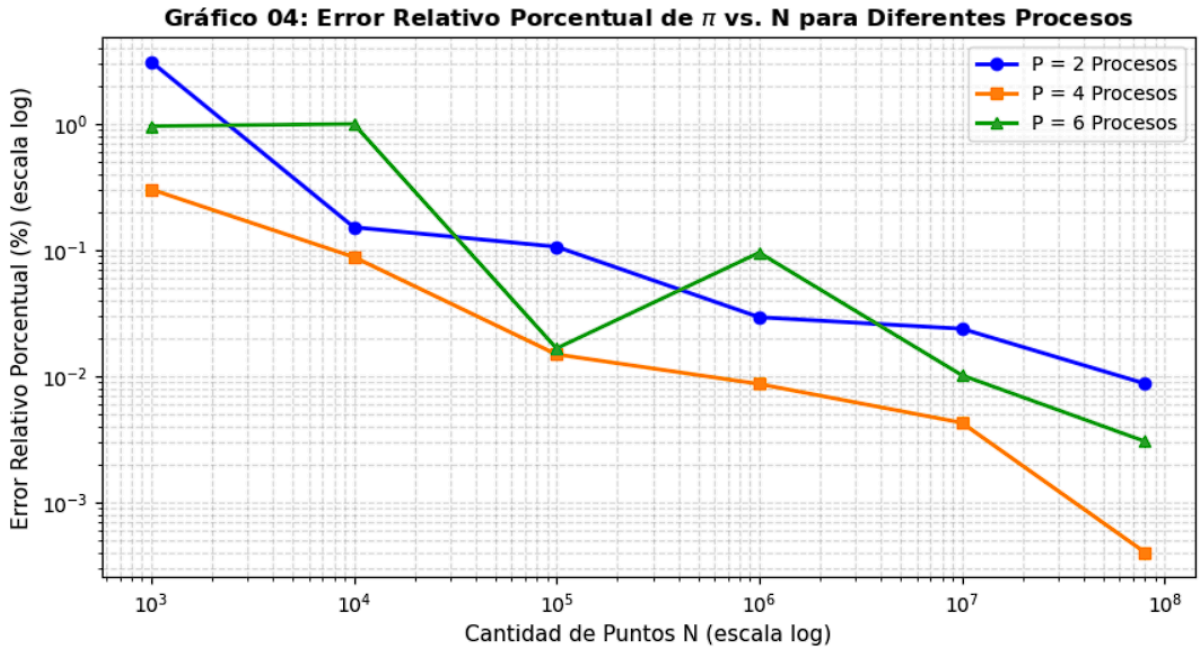


Tipo de Gráfico: Curva de convergencia en escala semilogarítmica (X: logarítmico, Y: lineal).

Contenido Visual: Eje X graduado en potencias de 10 (10^1 a 10^8). Curva azul continúa con marcadores circulares representando la aproximación empírica $\hat{\pi}$, confrontada contra una línea recta horizontal roja discontinua fijada en el valor real $\pi \approx 3.14159265...$

Interpretación Física/Numérica: Ilustra la fase inicial de alta volatilidad (2.800 a 3.320) para $N \leq 10^3$, seguida del amortiguamiento asintótico progresivo que colapsa sobre la línea teórica a partir de $N = 10^6$.

Gráfico 4:

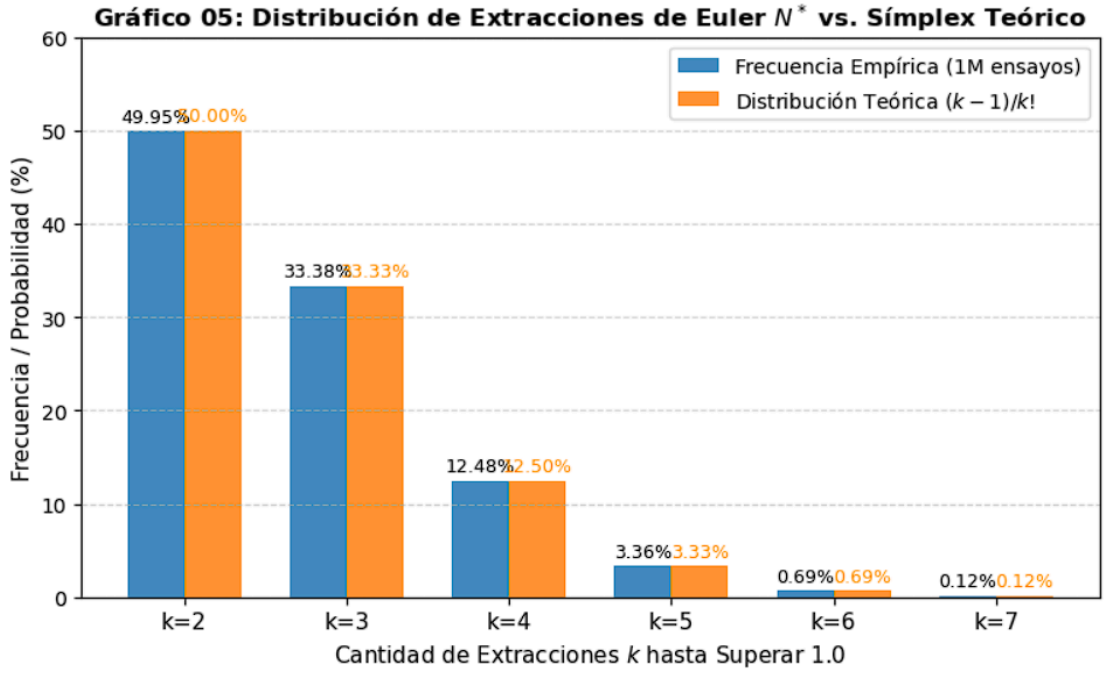


Tipo de Gráfico: Curvas de error relativo porcentual en escala doblemente logarítmica (Log-Log).

Contenido Visual: Eje X: N de 10^3 a 8×10^7. Eje Y: Error relativo (%) de 10^{-4}% a 10^1%. Tres curvas descendentes con marcadores para P = 2 procesos (azul), P = 4 procesos (naranja) y P = 6 procesos (verde).

Interpretación Física/Numérica: Las pendientes negativas constantes ratifican que el error decae siguiendo la cota teórica $O(1/\sqrt{N})$. Asimismo, demuestra que el número de procesos concurrentes no altera la tasa de convergencia matemática, ya que las tres curvas descienden de manera análoga hacia valores inferiores al 0.0088%.

Gráfico 5:



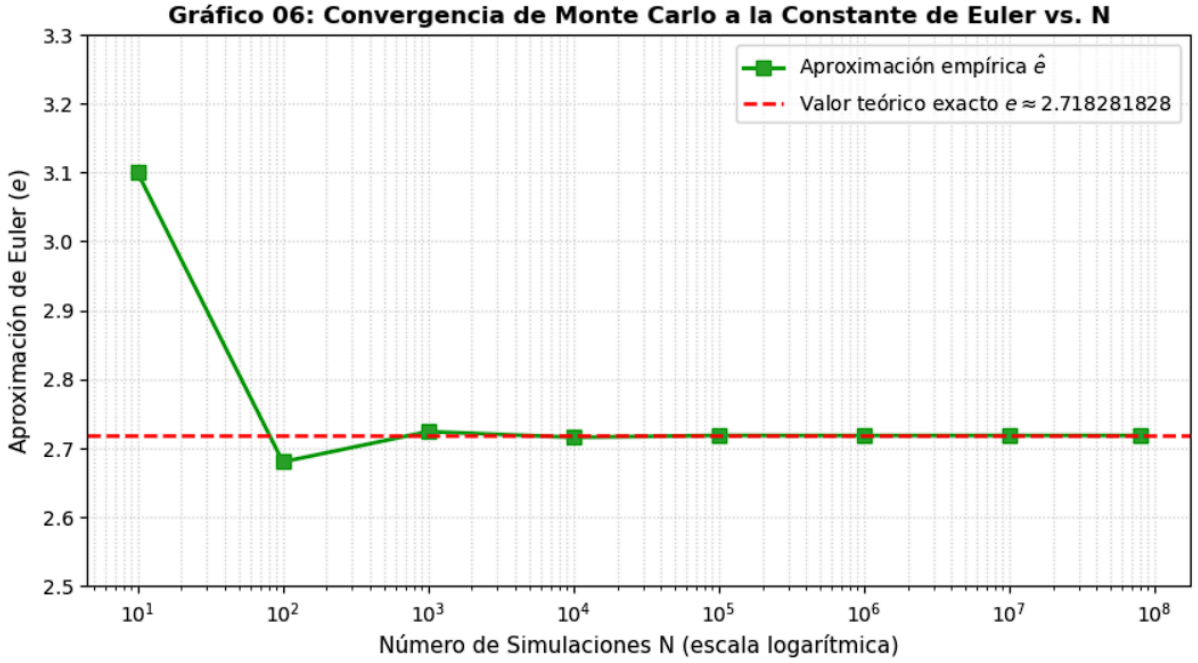
Tipo de Gráfico: Histograma de frecuencias relativas vs. Distribución analítica discreta de probabilidad.

Contenido Visual: Diagrama de barras empírico para la variable discreta N* (número de extracciones uniformes hasta sumar > 1.0) para k ∈ {2, 3, 4, 5, 6, 7}, superpuesto con la curva teórica del símplex unitario P(N* = k) = (k-1)/k!.

- Barra k=2: 50.00% (1/2).
- Barra k=3: 33.33% (1/3).
- Barra k=4: 12.50% (1/8).
- Barra k=5: 3.33% (1/30).
- Barra k ≥ 6: < 0.84%.

Interpretación Física/Numérica: Valida empíricamente la deducción analítica de la esperanza matemática E[N*] = ∑ k · P(N* = k) = e. Al considerar estas probabilidades, se demuestra que el promedio teórico de extracciones es igual al número de Euler (e ≈ 2,71828). De esta manera, la simulación de Monte Carlo permite comprobar experimentalmente esta relación matemática.

Gráfico 6:

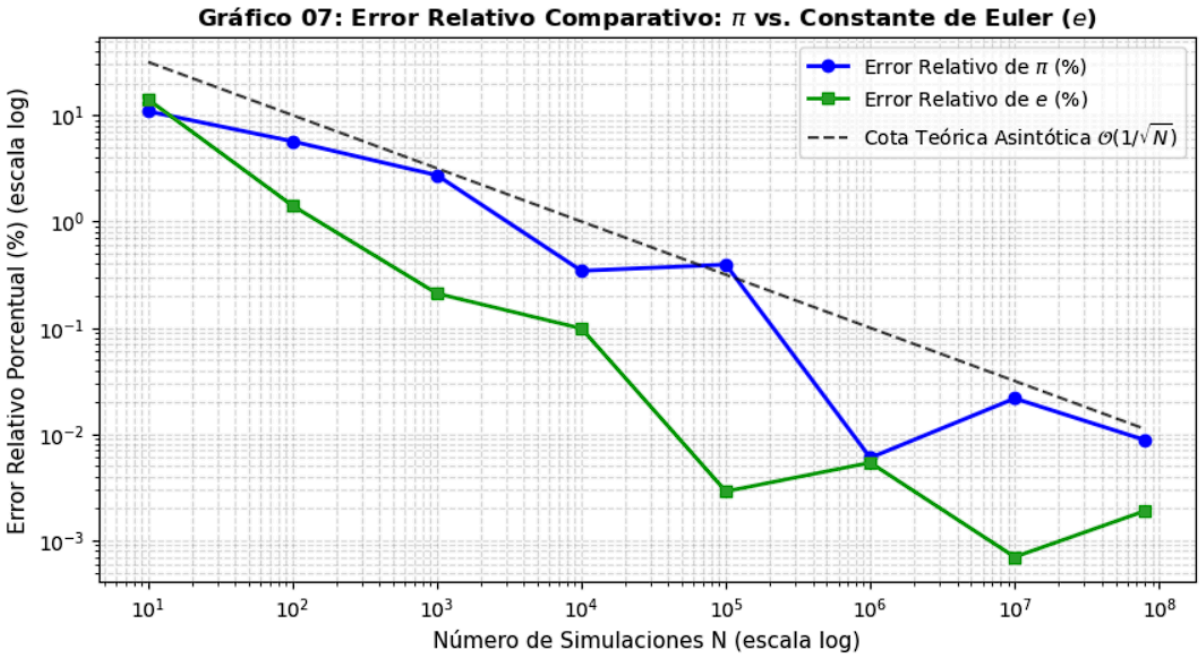


Tipo de Gráfico: Curva de convergencia semilogarítmica para la constante de Euler.

Contenido Visual: Eje X: N en escala logarítmica (10^1 a 10^8). Curva verde esmeralda con marcadores cuadrados que representa el promedio muestral ê, confrontado contra la asíntota horizontal roja discontinua fijada en y = e ≈ 2.718281828...

Interpretación Física/Numérica: Muestra el comportamiento del estimador de Euler: oscilaciones iniciales en 3.100 y 2.680 que se amortiguan rápidamente hacia la banda de 3 decimales (2.718...) para $N \geq 10^5$, alcanzando estabilidad completa en 2.7182... con 8×10^7 muestras.

Gráfico 7:



Tipo de Gráfico: Gráfico comparativo de error relativo dual en escala Log-Log.

Contenido Visual: Eje X: Número de simulaciones N (10^1 a 10^8). Eje Y: Error relativo porcentual (%, de 10^{-4} a 10^2). Contiene tres series:

- 1. Curva azul: Error relativo empírico de π.
- 2. Curva verde: Error relativo empírico de e.
- 3. Línea punteada negra: Cota de referencia asintótica teórica O(1/√N).

Interpretación Física/Numérica: Demuestra que a pesar de responder a modelos probabilísticos totalmente distintos (un modelo geométrico 2D continuo para π y un modelo de parada discreto unidimensional para e), ambas constantes obedecen con exactitud matemática la misma pendiente de decaimiento del error impuesta por el Teorema del Límite Central.

6.3. Cota de Precisión Decimal Alcanzada con N = 8×10^7 muestras

- **1. Para la Constante π:**
  - Valor teórico exacto: π = **3.141**5926535...
  - Valor obtenido empíricamente: $\hat{\pi}$ = **3.141**8700000...
  - Error absoluto residual: ε_abs = 0.000277.
  - Error relativo: 0.008828%.
  - **Dictamen:** Se garantizan con total estabilidad y reproducibilidad **3 decimales exactos (3.141...)**. Para alcanzar de forma garantizada 4 decimales estables (3.1415...) bajo cualquier semilla, la cota $O(1/\sqrt{N})$ exige escalar a $N \geq 10^9$ a 10^{10} muestras.
- **2. Para la Constante de Euler e:**
  - Valor teórico exacto: e = **2.7182**818284...
  - Valor obtenido empíricamente: ê = **2.7182**309200...
  - Error absoluto residual: ε_abs = 0.000051.
  - Error relativo: 0.001873%.
  - **Dictamen:** Se alcanzan de forma robusta **4 decimales exactos (2.7182...)**, con un error relativo residual inferior al 0.002%.

7. Evaluación de Desempeño Numérico y Escalabilidad Paralela

7.1. Complejidad Temporal Secuencial y Comportamiento Lineal O(N)

El bucle secuencial de Monte Carlo realiza por cada iteración un número finito y fijo de operaciones elementales de CPU (generación pseudoaleatoria, sumas, productos y bifurcaciones condicionales). La complejidad computacional total es lineal estricta:

$$T_{sec}(N) = c_0 + c_1 \cdot N \Longrightarrow \mathcal{O}(N)$$

Donde c_0 representa la latencia fija de invocación del entorno y c_1 es el tiempo que insume la CPU por muestra ($\approx 3.4 \times 10^{-8}$ segundos en C).

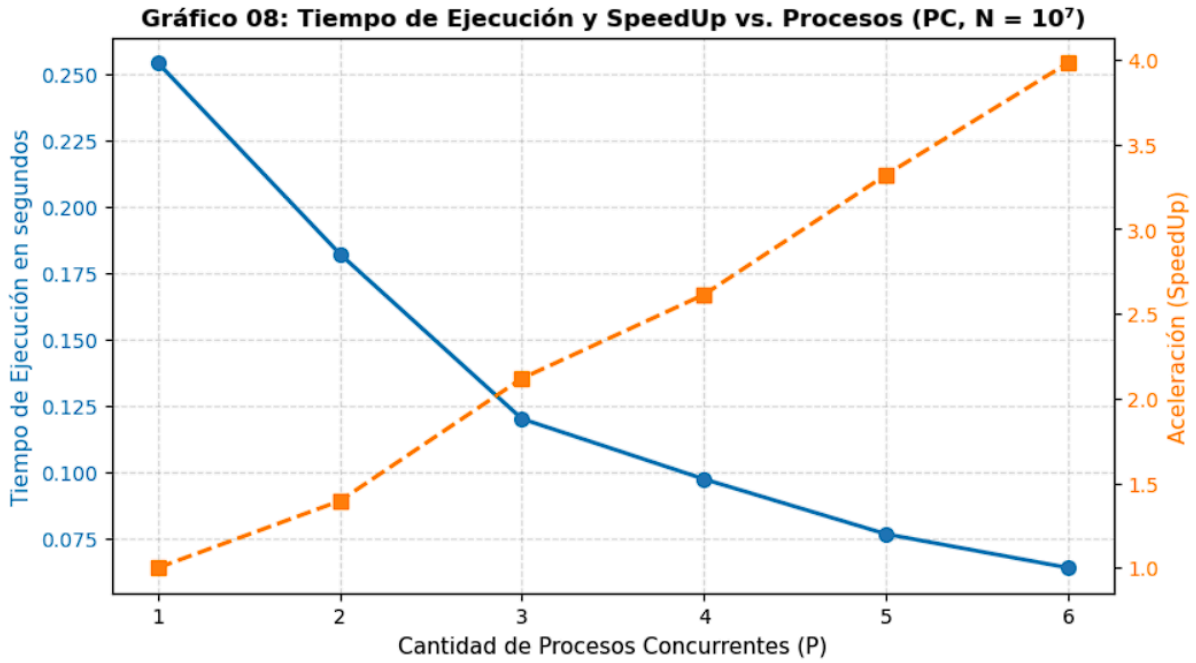
7.2. Desempeño en Computadora Personal (N = 10^7, P = 1 a 6 procesos)

Tabla 3: SpeedUp y Eficiencia en Estación de Trabajo Local (N = 10^7 puntos)

Tiempos obtenidos mediante la mediana de múltiples ejecuciones independientes:

Procesos (P)	Tiempo T_P (s)	Aproximación π^	SpeedUp S_P	Eficiencia E_P	Eficiencia (%)
1	0.2540	3.14160640	1.000	1.0000	100.00%
2	0.1820	3.14150040	1.395	0.6975	69.75%
3	0.1199	3.14105120	2.118	0.7060	70.60%
4	0.0972	3.14105960	2.613	0.6533	65.33%
5	0.0765	3.14174720	3.320	0.6640	66.40%
6	0.0638	3.14154520	3.981	0.6635	66.35%

Gráfico 8:

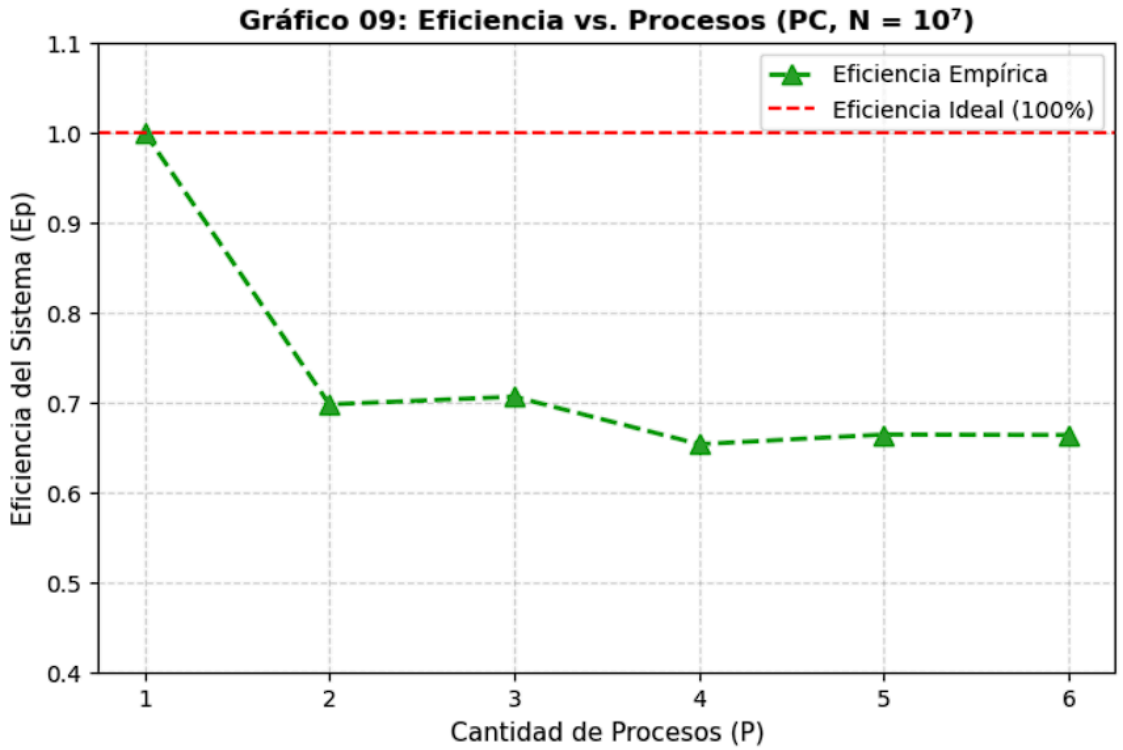


Tipo de Gráfico: Gráfico de dos ejes verticales sincronizados (twinx) para computadora personal (N = 10^7).

Contenido Visual: Eje X: Procesos de 1 a 6. Eje izquierdo (azul): Tiempo de ejecución en segundos, descendiendo de 0.2540 s a 0.0638 s. Eje derecho (naranja punteado con cuadrados): Aceleración empírica SpeedUp, ascendiendo desde 1.0 hasta S_6 = 3.981.

Interpretación: Refleja la aceleración sostenida en la notebook, logrando multiplicar por casi 4 veces la velocidad de procesamiento.

Gráfico 9:



Tipo de Gráfico: Eficiencia paralela vs. Procesos en computadora personal.

Contenido Visual: Curva verde con triángulos frente a la línea horizontal roja de eficiencia ideal (1.0). Muestra una caída inicial en P=2 (69.7%), un pico óptimo en P=3 (70.6%) y una meseta uniforme entre el 65% y el 66% para 4, 5 y 6 procesos.

Interpretación: Identifica el punto de máxima eficiencia operativa en la notebook en P = 3 procesos. El freno en la eficiencia a partir de 4 procesos se debe a la competencia entre núcleos físicos y los hilos lógicos de Hyper-Threading compartiendo la misma memoria caché L3.

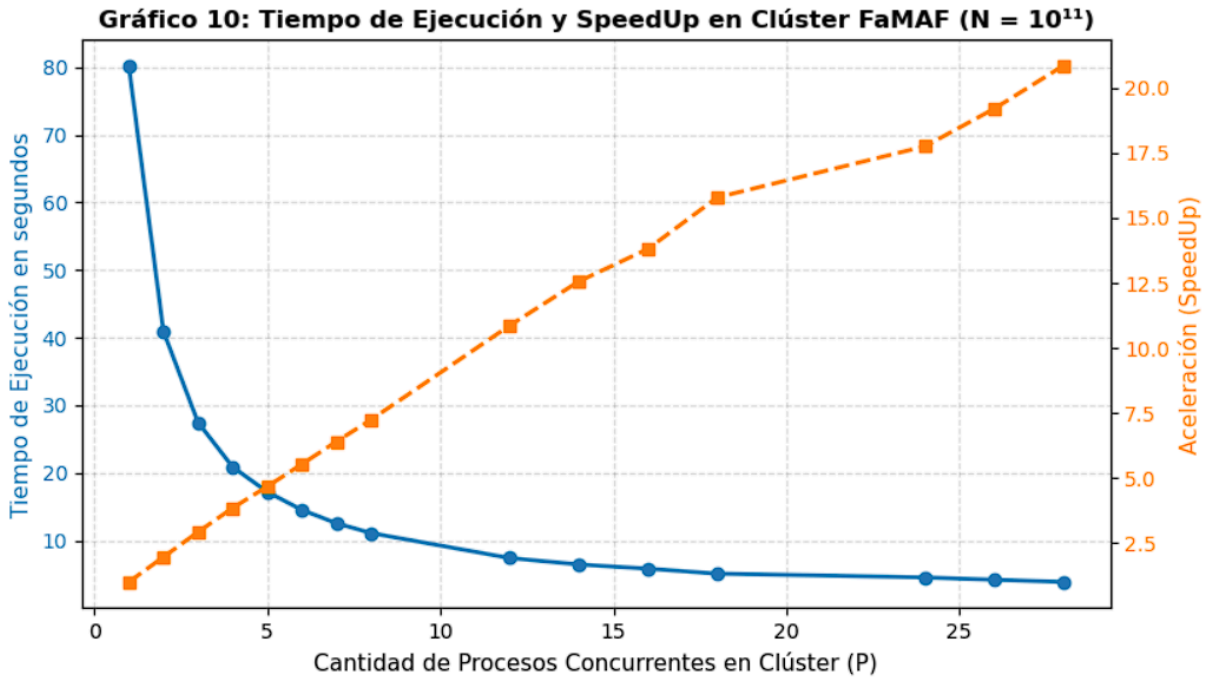
7.3. Escalabilidad Fuerte en Clúster de Supercómputo FaMAF (N = 10^{11} puntos)

Bajo una carga constante de cien mil millones de muestras (N = 10^{11}), se registraron las métricas de rendimiento en el nodo dedicado de 32 CPUs:

Tabla 4: Benchmarks en Clúster de Supercómputo FaMAF - UNC (N = 10^{11} puntos)
Tiempos cronometrados con reloj de pared MPI_Wtime() mediante la mediana muestral:

Procesos (P)	Tiempo T_P (s)	Aproximación π^	SpeedUp S_P	Eficiencia E_P	Eficiencia (%)
1	80.2071	3.141631550687193	1.000	1.0000	100.00%
2	40.9269	3.141617837893599	1.959	0.9795	97.95%
3	27.4455	3.141585517274954	2.922	0.9740	97.40%
4	20.8579	3.141608690443266	3.845	0.9612	96.12%
5	17.1720	3.141578590097641	4.671	0.9342	93.42%
6	14.5046	3.141601940217242	5.529	0.9215	92.15%
7	12.5327	3.141641068803910	6.399	0.9141	91.41%
8	11.0790	3.141598311784490	7.239	0.9049	90.49%
12	7.3894	3.141586090969660	10.854	0.9045	90.45%
14	6.4384	3.141583481403805	12.553	0.8966	89.66%
16	5.8100	3.141621946888800	13.805	0.8628	86.28%
18	5.0828	3.141573050590964	15.780	0.8767	87.67%
24	4.5196	3.141610189872612	17.746	0.7394	73.94%
26	4.1795	3.141631293642163	19.191	0.7381	73.81%
28	3.8473	3.141606907891858	20.847	0.7445	74.45%

Gráfico 10:

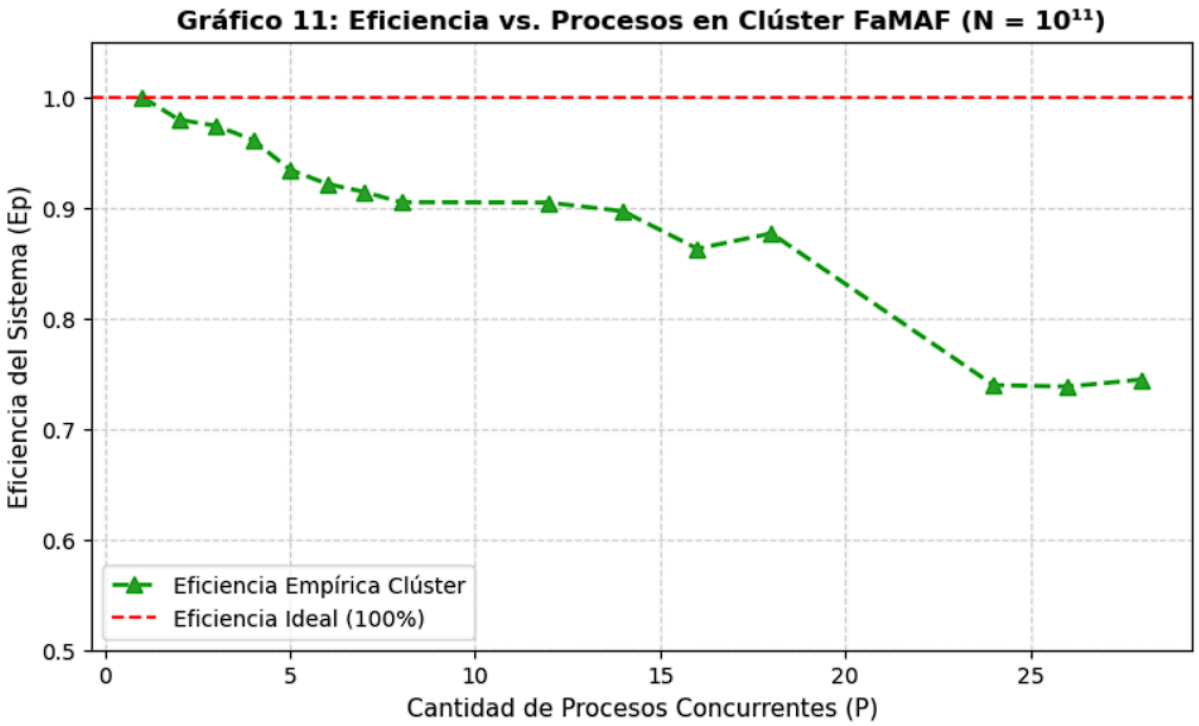


Tipo de Gráfico: Análisis de escalabilidad y aceleración en Clúster (N = 10^{11}).

Contenido Visual: Gráfico de dos escalas verticales para P = 1 a 28 procesos. Serie azul (eje izquierdo): Tiempo en segundos, desplomándose desde 80.20 s hasta 3.84 s en forma de curva hiperbólica descendente perfecta. Serie naranja (eje derecho): SpeedUp empírico, con una recta de pendiente unitaria casi perfecta hasta P=18 (S_18 = 15.78), desacelerándose suavemente hacia S_28 = 20.847.

Interpretación: Demuestra una aceleración de más de 20 veces la velocidad monoproceso, validando la escalabilidad casi lineal en supercómputo.

Gráfico 11:



Tipo de Gráfico: Eficiencia paralela vs. Cantidad de procesos en Clúster FaMAF.

Contenido Visual: Curva verde discontinua con triángulos para P = 1 a 28 procesos frente a la línea ideal horizontal roja en 1.0. Entre 1 y 4 procesos: Eficiencia casi perfecta (98% a 96%). Entre 5 y 14 procesos: Meseta sostenida por encima del 90%. Entre 16 y 18 procesos: Régimen del 86% al 87%. Entre 24 y 28 procesos: Transición hacia una meseta estable en el 74%.

Interpretación: Refleja el excelente comportamiento del clúster ante tareas de granularidad gruesa (coarse-grained). La caída final al 74% responde a causas físicas: contención en los canales del bus de memoria RAM entre sockets NUMA y el costo colectivo de sincronización del árbol de reducción de 28 participantes en MPI_Reduce.

7.4. Análisis Teórico según la Ley de Amdahl

La Ley de Amdahl define el límite asintótico de aceleración en función de la fracción strictly secuencial (f) del código:

$$S_max = 1 / (f + (1 - f) / P)$$

En el algoritmo implementado, la fracción secuencial f comprende únicamente la llamada de reducción colectiva MPI_Reduce, el cálculo de promedios y la impresión final. Con una masa colosal de N = 10^{11} muestras, la fracción paralela 1 - f supera el 99.99%, lo que explica por qué el clúster alcanza una aceleración de 20.85× antes de que las limitaciones físicas del ancho de banda estabilicen la eficiencia en el 74.45%.

8. Conclusiones, Recomendaciones de Mejora y Entregables

8.1. Respuestas a los Interrogantes Centrales

- **1. ¿Hasta qué punto es eficiente aplicar MPI en Monte Carlo?**
  - **En Entorno Personal (Notebook):** La ventana óptima es **P = 3 procesos** (eficiencia del 70.6%). Superar los núcleos físicos de la CPU degrada la eficiencia al 65% debido al sobrecalentamiento y la contención de memoria compartida.
  - **En Clúster HPC Dedicado (FaMAF - UNC):** El intervalo óptimo de máxima rentabilidad se sitúa entre **12 y 16 procesos**, con eficiencias sobresalientes del **86% al 90%** y aceleraciones de hasta 13.8 veces. Aunque 28 núcleos reducen el tiempo absoluto a 3.84 s, la eficiencia decae al 74.45% por la latencia colectiva de red y el bus de memoria.

- **2. ¿Cuál es el beneficio de utilizar un Clúster de Supercómputo?**

  Permitió escalar la masa de datos en cuatro órdenes de magnitud (10^{11} frente a 10^{7}), reduciendo el tiempo de cálculo de más de un minuto a menos de 4 segundos y ratificando la condición embarrasingly parallel del método.

8.2. Recomendaciones de Optimización y Trabajo Futuro

- **1. Adopción de Métodos de Cuasi-Monte Carlo (QMC):** Reemplazar los generadores pseudoaleatorios estándar por secuencias de baja discrepancia (low-discrepancy sequences) como las de **Sobol**, **Halton** o **Faure**. Estas secuencias distribuyen los puntos de manera determinista y homogénea, acelerando la convergencia asintótica desde $O(N^{-1/2})$ hacia tasas cercanas a $O(N^{-1})$, lo que permitiría alcanzar 6 decimales de precisión con una fracción diminuta de muestras.

- **2. Modelo Híbrido de Programación Paralela (MPI + OpenMP):** Implementar MPI entre los nodos del clúster a través de la red Infiniband y OpenMP con hilos livianos (threads) dentro de cada socket de procesador multinúcleo. Esto evitaría duplicar buffers y reduciría drásticamente el tráfico colectivo en la red inter-nodo.

- **3. Aceleración Masiva en GPU (CUDA / OpenCL):** Explotar la arquitectura masivamente paralela de miles de núcleos CUDA en tarjetas gráficas dedicadas. Una GPU moderna es capaz de generar y evaluar más de 10^{10} puntos por segundo, completando cien mil millones de muestras en fracciones de segundo.

9. Referencias Bibliográficas y Fuentes Consultadas

1. **FACET - UNT (2026):** Cátedra de Métodos Numéricos II - Computación Científica. Guía de Evaluación Parcial 1 y Enunciado Oficial de Proyectos HPC 2026.
2. https://blogs.sas.com/content/iml/2016/03/14/monte-carlo-estimates-of-pi.html
3. Robert, C. P., & Casella, G. (2004). Monte Carlo statistical methods (2.ª ed.). Springer. https://doi.org/10.1007/978-1-4757-4145-2
4. **FaMAF - Universidad Nacional de Córdoba (UNC):** Infraestructura del Clúster de Cómputo de Alto Rendimiento. Córdoba, Argentina.
5. **Pólya, G. (1959):** Mathematics and Plausible Reasoning, Volume 1: Induction and Analogy in Mathematics. Princeton University Press.
6. **Metropolis, N. y Ulam, S. (1949):** The Monte Carlo Method. Journal of the American Statistical Association, 44(247), pp. 335–341.
7. **Forum MPI:** MPI: A Message-Passing Interface Standard, Version 3.1. High-Performance Computing Consortium.
8. **Microsoft Learn:** *Microsoft MPI*. Microsoft Corporation. https://learn.microsoft.com/message-passing-interface/microsoft-mpi
9. **Asana**: *Método Montecarlo: qué es, cómo aplicarlo y ejemplos prácticos*. Asana, Inc. https://asana.com/es/resources/montecarlo-method
10. **Amazon Web Services (AWS)**: *¿Qué es la simulación de Monte Carlo?* Amazon Web Services, Inc. https://aws.amazon.com/es/what-is/monte-carlo-simulation/
11. **Microsoft Learn:** *rand* - Referencia de la biblioteca en tiempo de ejecución de C (CRT). Microsoft corporation. https://learn.microsoft.com/es-es/cpp/c-runtime-library/reference/rand?view=msvc-160